



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	MCA	
Course	Programming from First Principles	
Topic Name	Definitions: a means of looping indefinitely (practice session) Part 2	
Topic Code	-	
Unit/Module No. & Name	12	Recursion
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning and Definition	5
3.0	Structure Of A Recursive Function	6 - 8
4.0	Factorial Using Recursion	9 - 10
5.0	Summation Using Recursion	11 - 12
6.0	Fibonacci Using Recursion	13 - 14
7.0	Comparison Between Iteration And Recursion	15 - 16
8.0	Keywords Table	17 - 18

OBJECTIVE

1. This self-learning material explains recursion as an important concept in programming and problem solving.
2. It introduces the meaning of recursion, the structure of a recursive function, the role of base case and recursive case, and practical examples such as factorial, summation, and Fibonacci.
3. It also compares recursion with iteration so that students can understand when and why recursion is used. Recursion is valuable because it solves a large problem by repeatedly reducing it into smaller versions of the same problem.
4. This lesson uses simple language and academic explanation so that undergraduate students can build a clear foundation in the topic.
5. By the end of this unit, students should be able to define recursion, identify the structure of a recursive function, trace recursive calls, explain the importance of the base case, and distinguish recursion from iteration in Python programming with confidence and conceptual clarity.



1.0 INTRODUCTION

Recursion is one of the most important ideas in computer science because it helps programmers think about problems in a structured and logical way. In recursion, a function calls itself directly or indirectly to solve a smaller part of the same problem. This makes recursion useful for problems that naturally repeat in a similar pattern. Instead of solving the whole problem at one time, the function keeps reducing the task until it reaches a simple condition that can be solved directly. Many mathematical and algorithmic problems follow this pattern. In programming, recursion is used in factorial, Fibonacci series, tree traversal, graph processing, binary search, and divide-and-conquer algorithms such as quick sort and merge sort. Therefore, recursion is not only a theoretical topic but also a practical tool for solving important computational problems. Understanding recursion also improves a student's ability to think about structure, control flow, and function behavior in a deeper way.

1.1.1 Need For Studying Recursion

Students should study recursion because it teaches a different style of problem solving from loops and ordinary function calls. When a problem can be divided into smaller subproblems of the same type, recursion often gives a clear and elegant solution. It also helps students understand how function calls are stored and executed in memory through the call stack. This creates a stronger foundation for later topics such as trees, graphs, divide-and-conquer methods, and recursive data structures. In many advanced areas of programming, recursion appears naturally, so an early understanding of it becomes very valuable. In addition, recursive code is often shorter and easier to express than an iterative solution, especially when the problem itself has a recursive pattern. By studying recursion carefully, students develop better logical thinking, better tracing skills, and a stronger understanding of how programs solve repeated tasks step by step until a final result is reached.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Recursion

Recursion is a mathematical and programming concept in which a function solves a problem by calling itself again with a smaller or modified input. In simple words, recursion means that the function refers to itself to continue the solution process. The problem is not solved at once. Instead, it is divided into smaller parts, and each part is solved by the same function until a simple condition is reached. This makes recursion very useful for repeated and self-similar problems. The main idea is that a large task becomes easier when it is expressed in terms of a smaller version of itself. However, recursion must be designed carefully. If the function keeps calling itself without a proper stopping point, it may continue forever and cause failure. Therefore, recursion is a controlled form of repeated self-calling. It is a powerful concept because it combines simplicity of expression with deep problem-solving ability in both mathematics and programming.

2.1.1 Formal Definition Of Recursion

In academic terms, recursion is a programming and mathematical technique in which a function directly or indirectly calls itself to solve a problem by reducing it into smaller subproblems of the same form. A recursive solution generally contains a stopping condition and a self-referential step. The stopping condition, known as the base case, ensures that the repeated calls end. The self-referential step, known as the recursive case, continues the problem-solving process by passing a modified argument. This definition shows that recursion is not merely repeated calling. It is a structured process in which each function call moves toward a simpler instance of the same problem. In computer science, recursion is widely used because many computational structures, such as trees and recursive algorithms, naturally follow a self-similar form. Thus, recursion is both a conceptual and practical method for solving problems that are easier to express through repeated reduction than through direct iteration.

3.0 STRUCTURE OF A RECURSIVE FUNCTION

3.1 Base Case

The base case is the condition that stops recursion. It is one of the most important parts of any recursive function because it tells the function when to stop making further calls. Without the base case, the function would continue to call itself forever, leading to infinite recursion. In programming, such endless calling can exhaust available memory and crash the system. Therefore, the base case is necessary for both correctness and safety. The base case usually handles the smallest version of the problem, where the answer is already known directly. For example, in factorial, when n becomes 1, the answer is 1, so the function stops there. In summation, when n becomes 0, the answer is 0. The base case is like the final point of the recursive journey. Every recursive call must move toward this point. If the base case is properly defined, the function eventually stops and begins returning values correctly.

3.1.1 Importance Of The Base Case

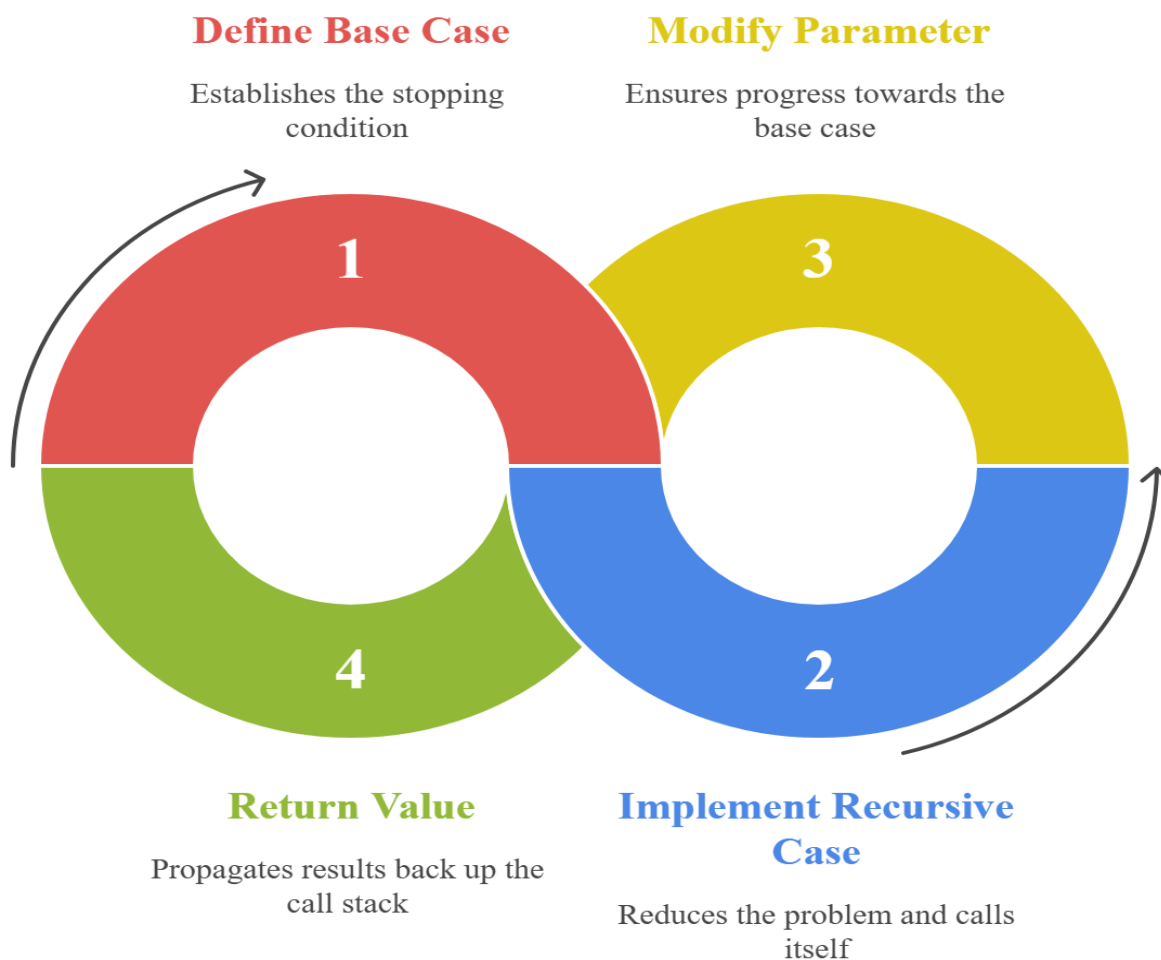
The importance of the base case lies in termination, correctness, and stability. First, it prevents the recursive process from continuing endlessly. Second, it provides a direct answer for the smallest form of the problem. Third, it allows the function calls to return backward in a meaningful order. If there is no base case, or if it is written incorrectly, then the program may go into infinite recursion. In Python, this can result in recursion depth errors or system failure because too many function frames are pushed onto the call stack. From a problem-solving point of view, the base case acts like the simplest truth on which the whole recursive logic is built. Every larger case depends on eventually reaching it. This is why the base case must be clear, reachable, and mathematically correct. In recursion, the quality of the solution depends greatly on the quality of the base case because it defines when the work is complete.

3.2 Recursive Case

The recursive case is the part of the function where the actual reduction of the problem takes place. In this step, the function does some work and then calls itself again with a smaller or modified parameter. This modified parameter is very important because it must move the

computation closer to the base case. The recursive case is what makes recursion dynamic and useful. It expresses the solution to the current problem in terms of a smaller version of the same problem. For example, factorial of n is written as n multiplied by factorial of n minus 1. Here, the recursive case does one multiplication and leaves the smaller task to the next function call. In summation, n is added to sum of n minus 1. Thus, the recursive case performs part of the solution and delegates the rest to the next recursive call. It is the engine of recursion because it keeps the process moving until termination occurs.

Fig. 1 Recursive Function Structure Cycle



This visual represents the recursive function cycle, highlighting defining the base case, implementing recursion, modifying parameters, and returning values as results propagate back through the call stack.

3.2.1 Modified Parameter In Recursive Functions

A recursive call must always use a modified parameter. This modification is what pushes the problem toward the base case. If the same parameter value is used again and again, the function will never progress and recursion will not stop. For this reason, the modified parameter is a central feature of recursive design. In factorial, n becomes n minus 1. In summation, n becomes n minus 1. In Fibonacci, the recursive case uses n minus 1 and n minus 2. These smaller values ensure that the function moves toward its stopping condition. The modified parameter shows that recursion is not random repetition. It is controlled progression. Every call changes the state of the problem so that it becomes easier to solve. This also helps students understand that recursion requires both logic and direction. A recursive function is correct only when the modification of the parameter guarantees that the base case will eventually be reached.

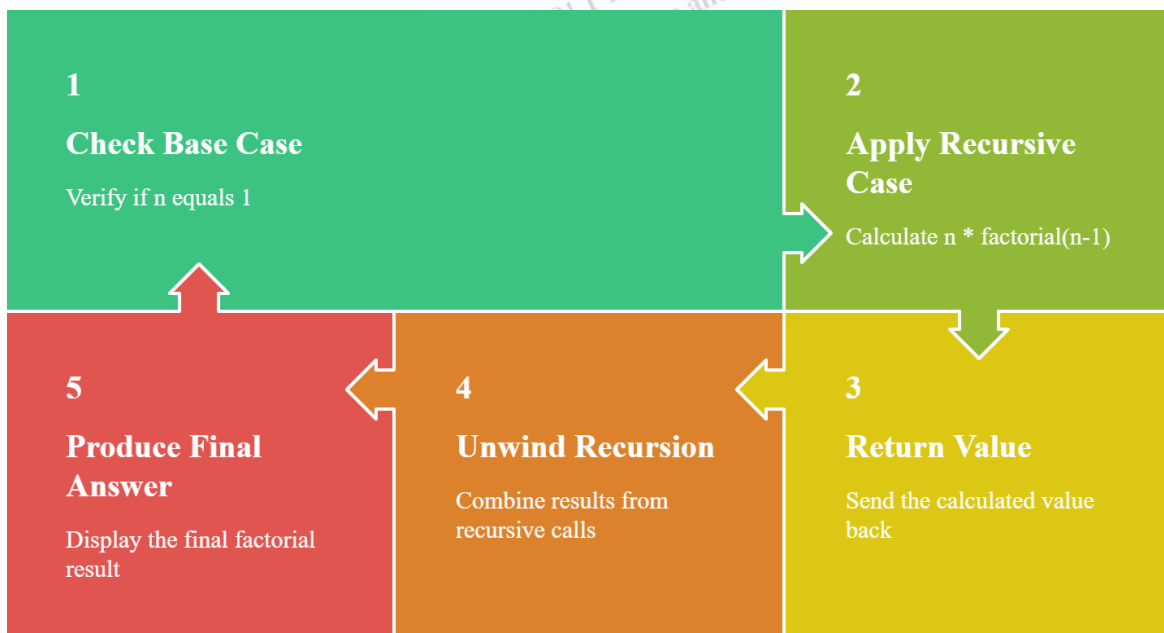


4.0 FACTORIAL USING RECURSION

4.1 Mathematical Representation Of Factorial

Factorial is one of the simplest and most common examples used to explain recursion. The factorial of a number n is the product of all positive integers from 1 to n . In mathematical recursive form, factorial of n is defined as 1 when n is equal to 1. Otherwise, factorial of n is equal to n multiplied by factorial of n minus 1. This representation naturally fits recursion because the answer for a large number depends on the same function applied to a smaller number. For example, factorial of 4 depends on factorial of 3, which depends on factorial of 2, which depends on factorial of 1. This chain continues until the base case is reached. Factorial is therefore a very good introductory example because it clearly contains both parts of recursion: a stopping condition and a repeated self-call using a reduced value. It helps students understand how recursion works mathematically before moving to Python code.

Fig. 2 Recursive Factorial Calculation Cycle



This diagram illustrates the recursive factorial process, showing steps from checking the base case, applying recursion, returning values, unwinding calls, and finally producing the computed factorial result.

4.1.1 Execution Sequence Of Factorial

Consider factorial of 3. The function first checks whether n is equal to 1. Since n is 3, the base case is not reached, so the recursive case is used. The function returns 3 multiplied by factorial of 2. Then factorial of 2 again checks the base case. Since n is not 1, it returns 2 multiplied by factorial of 1. Now factorial of 1 reaches the base case and returns 1. At this point, the recursive calls begin to return backward. Factorial of 2 becomes 2 multiplied by 1, which is 2. Then factorial of 3 becomes 3 multiplied by 2, which is 6. This backward returning process is sometimes described as unwinding recursion. It helps students understand that recursive calls build up in one direction and produce the final answer while returning in the opposite direction. This pattern is central to almost all recursive execution sequences.

4.2 Factorial Function In Python

In Python, the recursive factorial function is written with a normal function definition. The function begins by checking the base case using an if statement. If n is equal to 1, it returns 1. Otherwise, it uses the recursive case and returns n multiplied by factorial of n minus 1. After the function is defined, a value can be taken from the user and passed as an argument. The returned answer is then stored in a variable and printed. For example, if the user enters 3, the program will display that factorial of 3 is 6. This example is helpful because it connects mathematical thinking with practical coding. Students can see that the Python code directly follows the recursive mathematical structure. The factorial example is therefore useful not only for understanding recursion but also for learning how recursive ideas are written in an actual programming language like Python.

5.0 SUMMATION USING RECURSION

5.1 Mathematical Representation Of Summation

Summation is another simple and useful example for understanding recursion. It means adding all numbers from n down to 1. In recursive form, the sum function is defined as 0 when n is equal to 0. This is the base case. Otherwise, the recursive case says that sum of n is equal to n plus sum of n minus 1. This structure is very similar to factorial, but instead of multiplication, it uses addition. The example is valuable because it shows that recursion is not tied to only one mathematical operation. The same recursive pattern can be used in different kinds of problems. The summation function also makes it easy for students to trace the repeated calls and see how the answer is formed step by step. By using summation, learners understand that recursion is a general method for reducing a problem into smaller versions until a direct answer becomes possible.

5.1.1 Execution Sequence Of Summation

Let us consider sum of 4. The function first checks whether n is equal to 0. Since 4 is not the base case, it calls the recursive case and returns 4 plus sum of 3. Then sum of 3 returns 3 plus sum of 2. Sum of 2 returns 2 plus sum of 1. Sum of 1 returns 1 plus sum of 0. Now sum of 0 reaches the base case and returns 0. After that, the answers begin to return backward. Sum of 1 becomes 1 plus 0, which is 1. Sum of 2 becomes 2 plus 1, which is 3. Sum of 3 becomes 3 plus 3, which is 6. Finally, sum of 4 becomes 4 plus 6, which is 10. This sequence clearly shows how recursion builds a chain of calls and then resolves them from the smallest case upward to produce the final answer.

5.2 Summation Function In Python

The recursive summation function in Python follows the same style as factorial. The function definition begins by checking the base case. If n is equal to 0, it returns 0. Otherwise, the function returns n plus the result of calling the same sum function with n minus 1. This makes the code very close to the mathematical definition, which helps students understand it easily. After defining the function, it can be called with a chosen value such as 4. The returned answer

can then be displayed using a print statement. For example, sum of 4 returns 10. This coding example is useful because it shows that recursion works not only for multiplication-based problems but also for addition-based ones. It helps learners see that recursive thinking can be applied across many problem types, provided that a clear base case and proper recursive step are present in the function.



6.0 FIBONACCI USING RECURSION

6.1 Mathematical Representation Of Fibonacci

The Fibonacci function is different from factorial and summation because its recursive form contains two recursive calls instead of one. In the Fibonacci example used here, the base case says that if n is equal to 0 or 1, the function returns 1. Otherwise, the recursive case returns Fibonacci of n minus 1 plus Fibonacci of n minus 2. This makes the structure hierarchical rather than linear. In linear recursion, one call leads to one next call. In Fibonacci recursion, one call can branch into two more calls. This creates a tree-like pattern of execution. Fibonacci is an important example because it shows students that recursion can be simple or complex depending on how the recursive case is written. It helps learners understand branching recursion and prepares them for later topics such as tree structures and divide-and-conquer methods, where one call may expand into multiple recursive paths.

6.1.1 Execution Sequence Of Fibonacci

Consider Fibonacci of 3. The function first checks whether n is 0 or 1. Since n is 3, the base case is not reached. So the function returns Fibonacci of 2 plus Fibonacci of 1. Fibonacci of 1 immediately reaches the base case and returns 1. Fibonacci of 2 still requires recursion, so it returns Fibonacci of 1 plus Fibonacci of 0. Both of these values are base cases and each returns 1. Therefore, Fibonacci of 2 becomes 2. Then Fibonacci of 3 becomes Fibonacci of 2 plus Fibonacci of 1, which is 2 plus 1, giving 3. This sequence shows that Fibonacci recursion does not move in a straight line. Instead, it creates branches. Some calls reach the base case directly, while others continue expanding. This makes Fibonacci a good example of hierarchical recursion and helps students understand how recursive calls may combine results from more than one subproblem.

6.2 Fibonacci Function In Python

In Python, the recursive Fibonacci function is defined by first checking the base case using an if condition. If n is equal to 0 or n is equal to 1, the function returns 1. Otherwise, it returns the sum of two recursive calls: Fibonacci of n minus 1 and Fibonacci of n minus 2. This code is short and

directly reflects the mathematical definition. After the function is defined, it can be called with a value such as 3. The result returned by the function is then printed. In the example, Fibonacci of 3 gives 3. This program is useful because it helps students understand a more advanced kind of recursion than factorial or summation. At the same time, it also shows why recursion can become expensive in execution, since repeated calls may calculate the same values again. Thus, Fibonacci is both a teaching example and a bridge to deeper discussion about efficiency in recursive programming.



7.0 COMPARISON BETWEEN ITERATION AND RECURSION

7.1 Mode Of Implementation And State

Iteration and recursion are both used to repeat work, but they differ in how repetition is controlled. Iteration is implemented by using loops such as for and while. The repeated action depends on a control variable that changes in each step. In contrast, recursion is implemented by a function calling itself again and again with a modified parameter. In iteration, the state of the process is tracked through loop variables. In recursion, the state is stored in the call stack, where each function call has its own frame and parameter values. This difference is very important. Iteration is controlled from within one block of repeated statements, while recursion creates a chain of function calls. Both can solve repeated tasks, but recursion often mirrors mathematical definitions better, while iteration often feels more direct in simple programming tasks. Understanding this difference helps students choose the right method for a given problem.

7.1.1 Progression And Termination

In iteration, progression happens when the control variable moves toward the ending condition. For example, in a loop, the variable may increase from 1 to n or decrease in the opposite direction. The loop terminates when the condition becomes false. In recursion, progression happens when the parameter is modified in each function call so that it moves toward the base case. The recursion terminates only when that base case becomes true. This means both methods depend on movement toward a stopping point, but they do so differently. Iteration uses explicit loop control, while recursion uses repeated function calls with changing values. If the control variable does not change properly, iteration may become an infinite loop. If the parameter does not move toward the base case, recursion may continue endlessly and cause stack overflow. Thus, progression and termination are essential ideas in both approaches.

7.2 Code Size, Speed, And Practical Effects

Recursive code is often shorter, cleaner, and closer to the mathematical definition of a problem. This makes it elegant and easy to express in some cases. Iterative code is often longer because the programmer must handle loop control and intermediate states directly. However, in many

practical cases, iteration executes faster than recursion. This is because recursion creates a new function call frame each time, pushes it onto the stack, and later pops it when the value returns. This overhead takes extra time and memory. Iteration usually avoids that cost. When termination is missing, the effects also differ. In iteration, the program may enter an infinite loop and keep using CPU cycles. In recursion, repeated calls may exceed the recursion limit and cause stack overflow or crash. In Python, recursion depth is limited by default. Therefore, while recursion is elegant, iteration is often more efficient for straightforward repeated tasks.

1. Recursion is a method where a function calls itself.
2. A recursive function always needs a base case.
3. The recursive case reduces the problem to a smaller one.
4. Factorial is a standard example of linear recursion.
5. Summation is another linear recursive example.
6. Fibonacci demonstrates hierarchical or branching recursion.
7. Iteration uses loops, while recursion uses self-calling functions.
8. Recursion is often shorter and cleaner in expression.
9. Iteration is often faster in execution.
10. Missing termination causes infinite loops in iteration and stack overflow in recursion.

8.0 KEYWORDS TABLE

Keyword	Touch Vocabulary
Recursion	A function solving a problem by calling itself
Base Case	The stopping condition of recursion
Recursive Case	The self-calling step with reduced input
Modified Parameter	The changed value passed in the next call
Factorial	Product of integers from 1 to n
Summation	Addition of integers from 1 to n
Fibonacci	A branching recursive sequence
Call Stack	Memory area storing active function calls
Iteration	Repetition using loops

Infinite Loop	Endless repetition in iteration
Stack Overflow	Too many recursive calls causing failure
Linear Recursion	Recursion with one main recursive call
Hierarchical Recursion	Recursion with branching calls
Divide And Conquer	Solving problems by splitting them
Algorithm Design	Planning step-by-step problem solutions

